COMP 018: Database Administration — Activity 3 Progress Report

System Development, Repository Deployment, and Progress Documentation

From Database Design to a Working System

Cover Page & Metadata

Field	Details
Course	COMP 018 — Database Administration
Activity	Activity 3: System Development and Repository Deployment
Project Name	**CampusSafe: Campus Hazard & Safety Reporting System**
Evaluating / Submitting Group	**Group 7:** Jefferson Orpesa (Lead), Alba, Arreza, Robiso, Santos
Repository URL	https://github.com/jeffersonorpesa2-ops/grp07-campussafe
Tech Stack	PHP 8.2+ (MVC-lite, native PDO), PostgreSQL 14+ / 18.6, HTML5/CSS3/Vanilla JS
Branch Tested	`main`
Class Google Sheets Tracker	Registered in Activity 3 Master Registry & Activity 3 Testing Log

---

1. System Overview & Scope

1.1 Purpose & Scope

CampusSafe is an enterprise-grade campus hazard, incident, and safety reporting platform designed for university environments. Built directly from our approved Activity 2 relational schema, CampusSafe bridges the communication gap between the campus community (students, faculty, administrative staff) and facilities maintenance personnel.

The platform provides a transparent, auditable **6-stage hazard lifecycle**:

```
[REPORTED] ──> [VERIFIED] ──> [ASSIGNED] ──> [IN_PROGRESS] ──> [RESOLVED] ──> [CLOSED]
```

1.2 Core Functional Modules

1. **Configurable Connection Engine:** Robust PDO singleton supporting `.env` parameterization for custom hosts, ports (`5432`/`5433`), databases, and credentials.

2. **Session Authentication & RBAC:** Secure bcrypt password hashing, session fixation prevention (`session_regenerate_id`), and server-side role gating (`Auth::requireRole`).
3. **Hazard Reporting Module:** Dynamic report filing with categorized dropdowns, location mapping, input validation, and transactional history recording.
4. **Photo & Attachment Storage:** Secure multi-format file uploads (JPG, PNG, WEBP, PDF up to 5MB) with MIME verification and randomized hashes.
5. **Admin Hazard Verification:** Real-time verification queue where administrators inspect submitted hazards, assign priority (`Low`, `Medium`, `High`, `Critical`), and record findings.
6. **Maintenance Dispatch Queue:** Dispatch interface allowing administrators to assign verified hazards to active maintenance personnel with work instructions.
7. **Technician Portal & Work Orders:** Dedicated interface for technicians to accept assignments, log repair actions, and submit resolution timestamps.
8. **Audit Timeline & Multi-Criteria Search:** Real-time search/filter (status, priority, category, location, date range) with a chronological lifecycle history timeline.
9. **Interactive Safety Statistics:** Live dashboard visualizing hazard distribution by lifecycle status, severity level, category, and building.
10. **Administrative Master Data CRUD:** Management suites for users, hazard categories, and campus locations with soft deactivation (`is_active` / `account_status`).

---

2. Application User Roles & Database Mapping

Inside CampusSafe, human user roles are strictly segregated from PostgreSQL database roles:

Application Role	Responsibilities	Functions & Data Access	Intended Database Role
Administrator (`demo_admin`)	Full system oversight and dispatch	Verifies reports, assigns technicians, manages master CRUD (`users`, `categories`, `locations`), views safety analytics	Elevated administrative group role (`campussafe_admin_grp`), non-superuser
Maintenance Lead (`demo_maint1`)	Work order delegation and execution	Accepts jobs, updates status to `in_progress`, logs repair actions, marks hazards `resolved`	`app_runtime_grp`
Maintenance Tech (`demo_maint2`)	Field repairs and maintenance logging	Performs physical repairs, attaches resolution notes and parts logs	`app_runtime_grp`
Student (Reporter) (`demo_student`)	Hazard discovery and report submission	Submits new hazard reports, uploads photos, inspects status of own submissions	`app_runtime_grp` (row-filtered by application)
Faculty (Reporter) (`demo_faculty`)	Academic facility hazard reporting	Submits classroom and lab hazard reports, tracks resolution progress	`app_runtime_grp` (row-filtered by application)
Safety Auditor (`demo_auditor`)	Safety compliance and inspection	Read-only inspection of safety reports, audit timelines, and incident statistics	`app_readonly_grp`

Application Role	Responsibilities	Functions & Data Access	Intended Database Role
Application Runtime (System)	Connection account used by PHP PDO	Executes authorized SELECT, INSERT, UPDATE queries required by controllers	`app_runtime_grp` via login `campussafe_app`

---

3. Development Progress Log & Team Split

Day / Milestone	Phase	Description	Responsible Member	Key Commit Reference	Status
Day 1	Phase 1	Repository scaffolding, `.gitignore`, `.env.example`, directory hierarchy	Santos	`feat: initialize project scaffolding and env template`	COMPLETED
Day 1	Phase 2	Schema DDL, table definitions, foreign keys, cascade rules, constraints	Jefferson Orpesa	`feat: implement 01_schema.sql with constraints and indexes`	COMPLETED
Day 1	Phase 2	Seed data generation covering all 6 lifecycle states and realistic campus incidents	Jefferson Orpesa	`feat: add 02_seed.sql and dynamic user seeder`	COMPLETED
Day 2	Phase 3	Database PDO singleton, lightweight environment loader, CSRF validation engine	Alba	`feat: implement Database singleton and CSRF security guard`	COMPLETED
Day 2	Phase 3	Session authentication, bcrypt password hashing, role-based authorization	Alba	`feat: add Auth session manager with RBAC guards`	COMPLETED
Day 2	Phase 3	Hazard report submission form with category/location dropdowns & file upload	Alba & Robiso	`feat: build report submission form with validation`	COMPLETED
Day 3	Phase 3	Admin verification queue, priority assignment, verifier pair constraints	Arreza	`feat: implement admin verification and priority workflow`	COMPLETED
Day 3	Phase 3	Maintenance dispatch queue, technician assignment logic, assignment constraints	Arreza	`feat: create maintenance assignment dispatch module`	COMPLETED
Day 3	Phase 3	Maintenance technician work orders, action logging, resolution timestamping	Robiso	`feat: add maintenance action logging and resolution form`	COMPLETED

Day / Milestone	Phase	Description	Responsible Member	Key Commit Reference	Status
Day 4	Phase 3	Advanced search and filter, date range filter, student self-service isolation	Robiso	`feat: build search, filtering and report detail timeline`	COMPLETED
Day 4	Phase 3	Master data CRUD for users, categories, and campus locations with soft deactivation	Arreza	`feat: implement administrative master data management`	COMPLETED
Day 4	Phase 3	Safety analytics dashboard with KPI cards and distribution progress bars	Robiso	`feat: add safety statistics and analytics dashboard`	COMPLETED
Day 5	Phase 4	1-Click web installer wizard (`install.php`), `setup.bat`, and `start.bat` launchers	Jefferson Orpesa	`feat: add 1-click web installer wizard and launchers`	COMPLETED

---

4. Database Implementation & Schema Details

4.1 Relational Architecture (9 Core Tables)

- **`role`:** 5 system roles (`Student`, `Faculty/Staff`, `Maintenance`, `Administrator`, `Auditor`).
- **`app_user`:** User profiles, contact emails, and bcrypt password hashes.
- **`hazard_category`:** Catalog of hazard types (Electrical, Structural, Chemical, Fire Safety, Slip/Trip/Fall, Sanitation).
- **`campus_location`:** Campus buildings, floor levels, zones, and GPS coordinates.
- **`hazard_report`:** Core hazard reports with 6 lifecycle statuses (`reported`, `verified`, `assigned`, `in_progress`, `resolved`, `closed`) and 4 priorities (`Low`, `Medium`, `High`, `Critical`).
- **`assignment`:** Work order dispatch records linking verified hazards to maintenance technicians.
- **`maintenance_action`:** Granular repair action logs, parts used, and work notes.
- **`attachment`:** Photographic and document evidence with MIME verification.
- **`report_status_history`:** Immutable chronological audit trail recording every lifecycle transition and acting user.

4.2 SQL Scripts & Migrations

- `database/01_schema.sql`: Complete DDL table schema, foreign keys with referential actions, and custom CHECK constraints (`chk_verified_pair`, `chk_resolved_has_date`, `chk_status_valid`).
- `database/02_seed.sql`: Relational seed dataset populating 6 user accounts, 6 categories, 8 locations, 6 incident reports spanning all lifecycle states, work orders, repair logs, and attachments.
- `database/99_reset.sql`: Clean re-initialization script.
- `database/seed_users.php`: Dynamic PHP script ensuring cryptographically secure bcrypt hashes (`Password123!`).

4.3 Design Adjustments from Activity 2

Design Change	Rationale
Table `user` renamed to `app_user`	Avoids collision with PostgreSQL's reserved SQL keyword `USER`.
Added dedicated `report_status_history` table	Activity 2 tracked status only on `hazard_report`. An explicit history table guarantees a tamper-proof audit trail of timestamps and users.
Added CHECK constraint `chk_verified_pair`	Enforces data integrity: `verified_by_user_id` and `verified_at` must either both be NULL or both be populated.
Added CHECK constraint `chk_resolved_has_date`	Enforces that reports marked `resolved` or `closed` must have a non-null `resolved_at` timestamp.

---

5. Application Evidence & Verification

CampusSafe is fully functional and tested end-to-end:

1. **Zero-Effort 1-Click Setup (`install.php`):**
2. Enables classmates to initialize the database in one click from their browser without running CLI commands or manually configuring `.env`.
3. **Authentication & Session Switching:**
4. Features a convenient demo switcher on the login screen allowing instant role testing as Admin, Tech, Student, Faculty, or Auditor without manual password typing.
5. **Report Submission & File Uploads:**
6. Validated report submission across categories and campus locations with photo attachments.
7. **Verification & Work Order Dispatch:**
8. Admin verification queue correctly transitions status from `reported` to `verified`, assigns priority, and dispatches to maintenance leads.
9. **Technician Resolution & Action Logging:**
10. Technician portal successfully accepts assignments, logs repair notes, and closes tickets with resolution timestamps.
11. **Master Data CRUD Suites:**
12. Full Create, Read, Update, and Soft-Deactivate interfaces for Users, Hazard Categories, and Locations.

---

6. Setup and Configuration Guide

6.1 Automated Setup (Zero Effort)

1. Double-click `start.bat` in the project root.
2. The browser automatically navigates to `http://localhost:8000/install.php`.
3. Confirm PostgreSQL settings and click "🚀 **Run 1-Click Installation & Seed Database**".
4. The system initializes the database, loads seed data, writes `.env`, and launches the portal.

6.2 Manual Setup

1. Copy `.env.example` to `.env`: `bash cp .env.example .env`
2. Configure your PostgreSQL port (`5432` or `5433`) and credentials in `.env`.
3. Execute SQL scripts via `psql`: `bash psql -U postgres -d grp07_campussafedb -f database/01_schema.sql psql -U postgres -d grp07_campussafedb -f database/02_seed.sql`
4. Start the application: `bash php -S localhost:8000 -t public`

---

7. Issues Encountered & Technical Resolutions

#	Issue Encountered	Root Cause	Technical Resolution Applied
1	`Database Connection Error: could not find driver (SQLSTATE[08006])`	PHP CLI launched without PostgreSQL PDO drivers loaded.	Created `php.ini`, enabled `extension=pdo_pgsql` and `extension=pgsql`.
2	Password authentication failure on `postgres` user	Template `.env` password mismatched local machine configuration.	Parameterized `.env` to match the local PostgreSQL master password.
3	Port conflict between PostgreSQL 14 and 18 (5432 vs. 5433)	Workstation had multiple PostgreSQL instances running simultaneously.	Parameterized `DB_PORT` in `.env` and connection singleton.
4	Broken attachment thumbnails in report view	Relative upload paths were misaligned with the web root.	Created dedicated attachment handler in `src/helpers/functions.php` and public upload symlink.
5	Sequence permission denied when inserting hazards	Granting table privileges did not grant sequence usage in PostgreSQL.	Added explicit grant: `GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO app_runtime_grp;`.
6	Referential integrity block when deleting users	Foreign keys to historical reports prevented physical deletion.	Implemented soft deactivation (`account_status = 'inactive'`) across all administrative user management interfaces.

---

8. Cross-Group Peer Deployment & Verification Summary

Group 7 cloned, initialized, and tested the Activity 3 codebases of all **8 other groups** listed in the class Google Sheets tracker:

Tested Project	Cloned Repository URL	Setup Result	Documented Roles Exercised	Issues Identified & Feedback Sent
Group 4 (BrewPOS)	`https://github.com/Nycri/brewpos`	Success	Cashier, Manager, Admin	Spring Boot / JPA schema verified; noted default `postgres` username in `.env`.
Group 2 (Park-It)	`https://github.com/HelloP0/PARK-IT-DATABASE-MANAGEMENT-SYSTEM`	Success	Driver, Parking Attendant, Admin	Synthesized relational dataset to resolve minimal seed data; verified parking transactions.
Group 3 (OJT RMA)	`https://github.com/swvenz/Project-OJT-RMA-System.git`	Success	Student, Supervisor, Admin	Discovered submitted SQL dump contained `lending` schema instead of `ojt_managementdb`; reported to owners.
Group 5 (Lost & Found)	`https://github.com/grp05-dba/grp05_lostandfound`	Success	Student, Admin, Staff	Python / Flask setup; verified claim workflows and public report listings.
Group 6 (HabalGo)	`https://github.com/jeanaroyo/HabalGo`	Success	Passenger, Driver, Admin	Node.js / Express backend; verified ride booking and driver fare logging.
Group 8 (Campus Canteen)	`https://github.com/EclipseChan/bwisIT-Solutions---Campus-Canteen-Pre-Order-and-Queue-Management-System`	Success	Student, Stall Vendor, Admin	Verified pre-order cart checkout and vendor order fulfillment.
Group 10 (Restaurant)	`https://github.com/kathlenmaecallos11-arch/Restaurant-Online-Ordering-Table-Reservation-System`	Success	Customer, Staff, Admin	PHP backend; verified table reservations and anti-overlap constraint.
Group 11 (BoardTrack)	`https://github.com/karlrobertc-art/BoardTrack.git`	Success	Tenant, Caretaker, Landlord	Node.js / EJS; verified GiST lease exclusion constraint and room occupancy triggers.

---

9. Responsible AI Usage Declaration

- **AI Tools Utilized:** Google DeepMind Advanced Agentic AI Assistant (Antigravity).
- **Scope of AI Assistance:**
- Boilerplate generation for MVC-lite architecture and PDO transaction patterns.
- CSS styling for modern, accessible UI components, status badges, and responsive cards.

- Formulating automated setup batch scripts (`setup.bat`, `start.bat`) and testing scripts.
- **Human Verification & Operational Control:** All AI-generated code, SQL schema scripts, and authorization guards were inspected line-by-line, reviewed for SQL injection, CSRF vulnerabilities, and PostgreSQL 14+ compatibility, and verified through manual testing by Group 7 (Jefferson Orpesa, Alba, Arreza, Robiso, Santos).